Beta-Binomial Shrinkage for Sparse Response Prediction and the Limits of Item-Level Transfer

Olanrewaju Muili
School of Computing Sciences and Computer Engineering
University of Southern Mississippi
Hattiesburg, MS

olanrewaju.muili@gmail.com

Team ProbityX

Abstract

The Predictive AI Evaluation Competition asks for the probability that an AI system succeeds on each item of a benchmark on which it has never been evaluated, given at most 31 revealed outcomes per subject–benchmark pair, scored by the Brier area under the learning curve (ALC) at label budgets 0, 1, 3, 7, 15 and 31. Our submission is a Beta-Binomial shrinkage estimator: the success rate of each pair is estimated as $(s + m\mu)/(n + m)$ from s successes in n revealed labels, with an uninformative prior mean $\mu = 0.5$ and prior strength m. In a strict leave-one-benchmark-out (LOBO) evaluation on the 140 eligible pairs of the competition-provided training data, in which m is selected on three benchmarks and evaluated on the fourth, the estimator reaches a pooled equal-per-pair ALC of 0.1972, against 0.2210 for the official empirical-mean baseline and 0.2500 for a constant prediction. The submitted model fixes $m = 2$, which lies inside the range of fold-selected values; its ALC on the full training pool, 0.1956, is a descriptive figure reported separately from the cross-validated estimate. The gain is concentrated at budgets 1 and 3, where the empirical mean is overconfident. Under the same protocol we evaluated a configuration-aware prior mean, item difficulty predicted from item text, two acquisition policies and within-pair local smoothing. Some produced numerically better pooled ALC, but the gains depended on the one benchmark that contributes 82 of the 140 pairs or did not survive its exclusion, so the simpler estimator was submitted. An analysis-only oracle shows that interpreting labels relative to *true* item difficulty would reduce ALC to 0.121, locating the remaining headroom in transferable difficulty estimation rather than in the Bayesian update. The final Codabench submission (946829) passed the runtime check with a formative ALC of 0.170341 on a resampled hidden-test subset.

1 Introduction and task

The Predictive AI Evaluation Competition (PAIEC) at NeurIPS 2026 [8] asks whether the outcome of an AI evaluation can be predicted before it is run. A *subject* is a fixed AI configuration (model, harness, harness version, reasoning effort); an *item* is one task of a benchmark; a *response* is one run of a subject on an item, scored 0 or 1. The hidden test set is a *benchmark-level cold start*: its benchmarks appear nowhere in the training data. Under the published evaluation rules [8], for each subject–benchmark pair with at least 80 distinct items the platform splits the items at random into an *acquisition pool*, from which a submission may request labels, and an *evaluation pool*, which is scored; the split is fixed across submissions and budgets. A submission exposes `predict(input, labeled)`, which receives the subject, the item and the labels revealed so far and

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Eligible training benchmarks at the stated dataset revision. Pairs: subject–benchmark pairs with at least 80 distinct items, the unit the platform adapts on; Trials: mean recorded responses per subject–item cell; Success: overall response mean; Item chars: median character count of the item text.

Benchmark	Subjects	Items	Responses	Trials	Success	Pairs	Item chars
Multi-SWE-bench	82	2,126	57,975	1.00	0.148	82	1,147
REAL web agents	33	233	4,449	1.18	0.358	26	92
ResearchCodeBench	31	212	6,784	1.03	0.361	31	96,287
SWE-rebench	1	6,306	67,074	10.64	0.479	1	857
Total						140	

returns $\hat{p} \in [0, 1]$. Labels are revealed along one trajectory and predictions are scored at the nested budgets $n \in \{0, 1, 3, 7, 15, 31\}$. If no acquisition policy is supplied, the platform selects labels at random.

The Brier score [1, 3] at budget n, B_n, is the mean of $(\hat{p} - y)^2$ over all recorded responses on the pair's evaluation items. The *Brier ALC* is the area under the learning curve with budgets placed at $\log_2(n + 1)$, which the organizers state as

$$\mathrm{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}. \tag{1}$$

Pairs receive equal weight and lower is better. A constant prediction of 0.5 scores 0.25 at every budget. Eighty per cent of the weight falls on budgets 1 to 15, so the competition is decided by how a handful of labels is turned into probabilities. We claim no methodological novelty; the contribution of this report is a strict cross-benchmark evaluation that justifies a deliberately simple submission and documents why more structured alternatives were not adopted.

2 Data and evaluation protocol

2.1 Training data

All training data come from the competition-provided, gated `measurement-db` dataset [7] (Hugging Face `aims-foundations/measurement-db`), revision `bc8204d811823da849c6686bf124d4ca9f82e4de`. Of the six benchmark directories at that revision, four have binary item-level responses and are eligible under the competition's rule; MathArena (mixed response type) and MMDocRAG (fractional scores) were excluded. Table 1 summarises the eligible benchmarks.

Five properties of this pool shaped the work. **One dominant benchmark:** Multi-SWE-bench contributes 82 of the 140 pairs and SWE-rebench a single pair, so pooled statistics are driven by Multi-SWE-bench and every claimed gain is also reported without it. **Rare repeated trials:** only SWE-rebench has repeated runs per subject–item cell (10.6 on average); elsewhere fewer than 10% of cells have two or more, so the irreducible Brier floor $\mathbb{E}[\pi(1-\pi)]$ is identifiable only on SWE-rebench (0.051, against a base-rate Brier $\bar{y}(1 - \bar{y})$ of 0.250) and partially on REAL (0.082 on the 9% of cells with repeats). **Sparse configuration metadata:** harness is populated on Multi-SWE-bench (12 distinct values) and SWE-rebench (one value), harness version only on SWE-rebench, and reasoning effort on none; 17 of the 60 model names appear in more than one benchmark, so model identity is the only configuration attribute that overlaps across benchmarks. **Extreme variation in item text:** median item text runs from 92 characters (REAL) to 96,287 (ResearchCodeBench, whole papers with code); only the item text is present on every benchmark, the auxiliary item-feature string is benchmark-specific and the interactor field is empty everywhere. **No leakage:** no item content hash is shared across benchmarks, and no hidden benchmark examples or response labels were accessed (Section 6).

2.2 Offline evaluation protocol

We reimplemented the platform's test-time adaptation so that every modelling decision could be made offline under a benchmark-level cold start.

Leave-one-benchmark-out (LOBO). Each of the four benchmarks is held out in turn. Every fitted quantity used to predict on the held-out benchmark (hyperparameters, vocabularies, standardisation constants, prior means) is computed from the other three only, with hyperparameters chosen by *nested* LOBO inside those three. The *selection statistic* for every decision is the pooled equal-per-pair ALC over all 140 held-out pairs, not the mean of four fold averages, because the folds differ by two orders of magnitude in pair count; per-benchmark results are reported beside it.

Splitting, budgets and seeds. For every pair with at least 80 distinct items, the items are split 50/50 into acquisition and evaluation pools with a fixed split seed (0), matching the platform rule. Candidates from the acquisition pool are streamed in a seeded random order; an acquired item reveals one of its recorded responses; predictions on the evaluation pool are scored after 0, 1, 3, 7, 15 and 31 labels along the same trajectory, so the label sets are nested as on the platform. Candidate-order seeds 0, 1 and 2 are used for every protocol evaluation in Tables 2–4 and results are averaged over them. Nested inner hyperparameter selection (B3, B4 and H3 below) uses seed 0 only, as do the oracle and requirement analyses of Section 5.5; the difficulty-prediction screening of Section 5.2 is not a protocol run.

Metrics, uncertainty and tests. Per pair we compute B_n over all recorded responses, the expected calibration error (ECE) [4] with 10 equal-width bins weighted by response count, and the ALC of Eq. (1). Differences between methods are assessed with a paired bootstrap over pairs (5,000 resamples of the 140 per-pair ALC differences, each averaged over seeds; bootstrap seed 0), reported as the mean difference with a 95% percentile interval and the number of pairs improved. Forty unit tests accompany the code, including checks that the trapezoid ALC reproduces the weights of Eq. (1) and the illustrative curve on the competition page, that the split and label nesting match the platform, and that the submitted estimator with $m = 0$ recovers our port of the official empirical-mean predictor [9] exactly.

3 Submitted method: Beta-Binomial shrinkage

The official baseline (B1) [9] predicts, for a pair with s successes among n revealed labels, $\hat{p} = s/n$, and 0.5 when $n = 0$. After one label this is 0 or 1, which scores worse than the constant 0.5 whenever the pair's success rate lies strictly between 0 and 1: B1's budget-1 Brier on the training pool is 0.312, above the constant's 0.250 (Table 2).

The submitted predictor (B2) places a Beta prior on the pair success rate [2], $p \sim \mathrm{Beta}(m\mu,\, m(1-\mu))$, and predicts with the posterior mean

$$\hat{p} = \frac{s + m\mu}{n + m}, \qquad \mu = 0.5. \tag{2}$$

Labels are counted only if they belong to the same subject and the same anonymous benchmark identifier as the target item. B2 makes no assumption about the unseen benchmark's difficulty, the subject's identity or the item's text, and it converges to B1 as n grows.

Two evaluations of the prior strength m. We distinguish two figures throughout.

1. **Strict LOBO estimate (0.1972).** For each held-out benchmark, m is selected over the grid $\{0, 0.25, 0.5, 1, 1.5, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64, \infty\}$ ($m = 0$ is B1, $m = \infty$ the constant) by minimising pooled ALC on the pairs of the three *training* benchmarks, and the held-out benchmark is scored with that m^*: 3, 2, 1.5 and 2 for Multi-SWE-bench, REAL, ResearchCodeBench and SWE-rebench. This is the cross-validated estimate of the selection procedure.
2. **Fixed $m = 2$ (0.1956).** The submission freezes $m = 2$, a Beta(1,1) prior: the prediction is 0.5 with no labels and $2/3$ or $1/3$ after one label. The value lies inside the range of fold-selected strengths, and the in-sample curve of pooled ALC against m is flat between 1.5 and 3 (0.1957, 0.1956, 0.1967). Because $m = 2$ was fixed with all four folds visible, its full-pool ALC of 0.1956 is a *descriptive* figure, not an unbiased held-out estimate; the strict estimate of the procedure is 0.1972.

No acquisition policy is shipped (Section 5.3); the package is a single standard-library Python file with no state between calls and no learned parameters beyond the constants $\mu = 0.5$ and $m = 2$.

Table 2: Pooled equal-per-pair results on the training pool (140 held-out pairs; mean over candidate-order seeds 0, 1, 2; lower is better). B2 strict LOBO selects m on each fold's training benchmarks; B2 fixed $m = 2$ is the submitted model, evaluated descriptively. Lower block: mean ECE over pairs (10 bins).

Method	ALC	B_0	B_1	B_3	B_7	B_{15}	B_{31}
B0 constant 0.5	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500	0.2500
B1 empirical mean (official baseline)	0.2210	0.2500	0.3117	0.2157	0.1904	0.1762	0.1718
B2 strict LOBO (m^* per fold)	**0.1972**	0.2500	0.2187	0.1976	0.1841	0.1748	0.1714
B2 fixed $m = 2$ (submitted)	0.1956	0.2500	0.2150	0.1953	0.1826	0.1743	0.1712
Mean ECE over pairs							
B1 empirical mean		0.243	0.312	0.175	0.123	0.084	0.064
B2 strict LOBO		0.243	0.196	0.148	0.113	0.080	0.063
B2 fixed $m = 2$		0.243	0.189	0.140	0.105	0.077	0.062

Table 3: Per-benchmark ALC. m^* is the LOBO-selected prior strength for the fold. "Improved" counts pairs on which B2 strict LOBO has lower ALC than B1.

Held-out benchmark	Pairs	m^*	B1	B2 strict LOBO	B2 fixed $m = 2$	Improved
Multi-SWE-bench	82	3	0.2019	0.1832	0.1809	51 (62%)
REAL web agents	26	2	0.2340	0.2047	0.2047	21 (81%)
ResearchCodeBench	31	1.5	0.2571	0.2257	0.2244	29 (94%)
SWE-rebench	1	2	0.3257	0.2667	0.2667	1 (100%)
Pooled	140		0.2210	0.1972	0.1956	102 (73%)

4 Main results

Tables 2 and 3 and Figure 1 give the controlled evaluation on the training pool: B0 $= 0.2500$, B1 $= 0.2210$, B2 $= 0.1972$ (strict LOBO), and 0.1956 for the fixed-m submitted model.

Where the gain is (strict LOBO). The B1-minus-B2 Brier difference by budget under strict LOBO is $+0.093$ at budget 1, $+0.018$ at budget 3, $+0.006$ at budget 7, $+0.001$ at budget 15 and 0.000 at budget 31 (for the fixed $m = 2$ model: $+0.097$, $+0.020$, $+0.008$, $+0.002$, $+0.001$). The entire improvement is the repair of small-budget overconfidence, where Eq. (1) places 40% of its weight. Mean ECE at budget 1 falls from 0.312 to 0.196 under strict LOBO, and B2 is better calibrated than B1 at every positive budget.

Uncertainty. Under strict LOBO the paired bootstrap of the B2-minus-B1 ALC difference is -0.0238 with 95% interval $[-0.0295, -0.0180]$ and 102 of 140 pairs improved. For the fixed $m = 2$ model the corresponding figures, recomputed from the pair-level outputs for this report, are -0.0254 $[-0.0306, -0.0202]$ with 103 of 140 pairs improved. The gain holds on every benchmark (Table 3) and therefore without Multi-SWE-bench.

Where B2 loses. Stratifying the fixed-$m = 2$ results by the true pair success rate (an analysis-only split the predictor never sees), B2 is worse than B1 on the two lowest strata, ALC 0.087 versus 0.066 for rates in $(0, 0.05]$ (16 pairs) and 0.116 versus 0.108 for $(0.05, 0.1]$ (23 pairs), and better on the three higher strata (0.166 vs 0.179, 0.244 vs 0.289, 0.262 vs 0.319 for 21, 63 and 17 pairs): for a pair that almost always fails, a few zeros give B1 an estimate of 0 that beats B2's pull toward 0.5. This motivated the first alternative below. At budget 31 the fixed-m model (0.1712) is within 0.008 of the best predictor that is constant within a pair ($\bar{y}(1 - \bar{y})$ from each pair's success rate, 0.1635 pooled), so pair-level modelling is close to saturated at large budgets.

5 Alternative methods and ablations

All experiments here use the protocol of Section 2.2 with the fixed $m = 2$ model (pooled ALC 0.1956) as the reference, the model an alternative would replace; Table 4 collects the outcomes, with negative differences favouring the alternative.

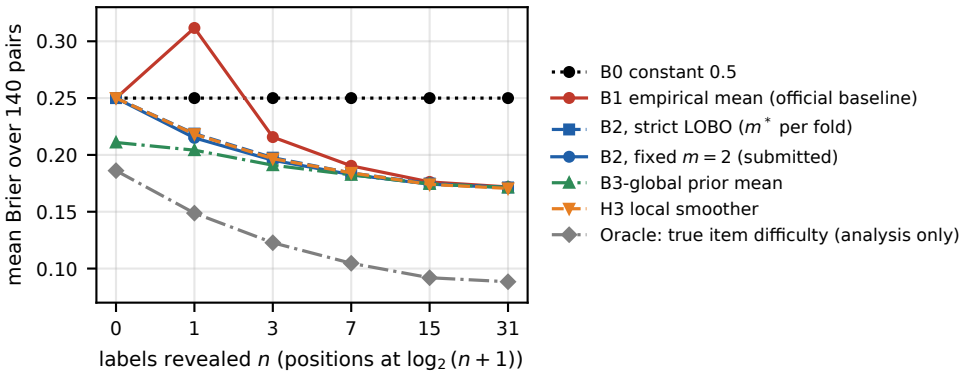


Figure 1: Learning curves on the training pool (mean Brier over the 140 pairs at each budget), from the pair-level outputs of the released scripts. The two B2 curves nearly coincide; B3-global and H3 are alternatives from Section 5; the oracle is analysis-only (Section 5.5, seed 0).

Table 4: Alternatives under the same protocol against B2 fixed $m = 2$ (ALC 0.1956; for the acquisition policies, the same model with random acquisition). Δ: paired-bootstrap mean ALC difference with 95% interval, negative favouring the alternative; Improved: pairs with lower ALC than the reference; last column: Δ on the 58 pairs outside Multi-SWE-bench (MSWE); last row analysis-only. Intervals not printed by the released scripts were recomputed from their pair-level outputs (Appendix A).

Method	ALC	Δ [95% CI]	Improved	Δ excl. MSWE [95% CI]
B3 configuration prior	0.1946	-0.0010 $[-0.0044, +0.0025]$	86/140	$+0.0023$ $[-0.0030, +0.0077]$
B3-global prior	0.1886	-0.0069 $[-0.0095, -0.0043]$	97/140	$+0.0020$ $[-0.0027, +0.0067]$
B4 text difficulty, nested λ	0.2009	$+0.0053$ $[+0.0032, +0.0075]$	62/140	$+0.0129$ $[+0.0082, +0.0175]$
B4 text difficulty, $\lambda = 1$	0.1969	$+0.0014$ $[+0.0001, +0.0027]$	75/140	$+0.0048$ $[+0.0025, +0.0073]$
B4 text level as prior mean	0.1981	$+0.0025$ $[-0.0012, +0.0064]$	90/140	$+0.0141$ $[+0.0061, +0.0225]$
H2 uncertainty sampling	0.1976	$+0.0020$ $[-0.0006, +0.0046]$	57/140	$+0.0058$ $[+0.0028, +0.0089]$
H2 eager labelling	0.1971	$+0.0016$ $[-0.0008, +0.0038]$	57/140	$+0.0056$ $[+0.0028, +0.0083]$
H3 local smoother	0.1967	$+0.0011$ $[+0.0004, +0.0019]$	99/140	$+0.0028$ $[+0.0010, +0.0047]$
B4, in-benchmark CV (analysis only)	0.1933	-0.0022 $[-0.0049, +0.0006]$	77/140	-0.0119 $[-0.0164, -0.0073]$

5.1 Configuration-aware prior mean (B3)

B2's losses are concentrated on very-low-success pairs because its prior mean is 0.5; a prior mean informed by the subject's configuration might repair them while keeping the small-budget calibration gain. The prior mean μ in Eq. (2) is replaced by a subject-specific estimate fitted on the training benchmarks: each training subject's success logit is centred on its benchmark's mean subject logit; the centred offsets are pooled at hierarchical back-off levels (exact configuration, model name, provider, harness) and shrunk toward zero with a constant c; and the prior mean is $\sigma(G + \text{offset})$, where G is the mean over training benchmarks of their mean subject logits. Prior strengths ($m \in \{1, 2, 3, 4, 6\}$ for informative and back-off levels) and $c \in \{0, 20, 100\}$ are selected by nested LOBO. The ablation *B3-global* drops the offset and gives every subject the prior mean $\sigma(G)$: 0.366, 0.307, 0.301 and 0.257 for the four folds.

Result. B3 matches B2 within noise (Table 4): it repairs the 37 pairs on which B2 lost to B1 (mean ALC 0.098 against B2's 0.103 and B1's 0.085) but pays on the high-success stratum (0.275 against 0.262 for rates above 0.5), and the two cancel. B3-global *does* produce a pooled improvement, -0.0069 $[-0.0095, -0.0043]$ with 97 of 140 pairs improved; but excluding Multi-SWE-bench the difference is $+0.0020$ $[-0.0027, +0.0067]$ on 58 pairs, it is worse than B2 on the high-success stratum (0.281 vs 0.262) and on the SWE-rebench pair (0.276 vs 0.267), and nested selection chose the heaviest shrinkage of the subject component ($c = 100$) on every fold.

Interpretation. Subject ability itself does transfer: the Spearman rank correlation of model-level relative ability on models shared between two benchmarks is $+0.58$ (Multi-SWE-bench vs REAL,

11 shared models), $+0.93$ (Multi-SWE-bench vs ResearchCodeBench, 9) and $+0.58$ (REAL vs ResearchCodeBench, 13). The prior's error is instead dominated by the unknown *benchmark level*: $\sigma(G)$ misses the held-out benchmark's mean pair success rate (0.220, 0.340, 0.353, 0.479) by $+0.146$, -0.033, -0.052 and -0.222 on the four folds. On Multi-SWE-bench, harness accounts for 0.76 of the between-pair variance of the success rate on the probability scale, versus 0.21 for model name; on the logit scale the harness share is 0.77. Harness is unpopulated elsewhere, so it cannot be learned across benchmarks. The B3-global gain is therefore the fact that Multi-SWE-bench is hard and 0.5 is too optimistic for it: a bet that hidden benchmarks resemble the training pool's difficulty mix, which a hidden benchmark with high success rates would punish, and not robust enough to replace B2 (provenance of these diagnostics: Appendix A).

5.2 Item difficulty from item text (H1)

Few-shot prediction should improve if labels are interpreted relative to item difficulty rather than as exchangeable draws: failing an easy item is stronger evidence of low ability than failing a hard one. Model B4 keeps a posterior over pair ability θ on a grid with Rasch-type likelihood $\sigma(\theta - \lambda\delta_i)$ [5] and a uniform prior on the success probability; $\lambda = 0$ matches B2 to the grid tolerance of the unit test (about 2×10^{-3}). The relative difficulty δ_i is predicted from item text by ridge regression fitted on the training benchmarks only, with targets centred within benchmark and the penalty chosen by nested LOBO. We ask whether text predicts difficulty out of benchmark at all, run B4 in the protocol, and screen four representations: 19 hand-crafted text statistics, TF-IDF with a training-fold vocabulary, and the frozen encoders all-MiniLM-L6-v2 and bge-base-en-v1.5 [6, 10, 11] with deterministic 1,500-character chunking and mean pooling, against a gate fixed before the run: pooled out-of-benchmark Spearman correlation of at least 0.50 with no fold below -0.10.

Result. With the hand-crafted features, the out-of-benchmark Spearman correlation between predicted and true relative difficulty is $+0.05$, $+0.21$, -0.36 and $+0.06$ on the four folds (the same features reach 0.20 to 0.39 within a benchmark under in-benchmark cross-validation), and no feature keeps a stable sign across benchmarks. Aggregated text predictions also fail to estimate an unseen benchmark's level (ResearchCodeBench 0.10 against a true 0.36; SWE-rebench 0.13 against 0.48) and are no better than the training-benchmark mean where the domains match. In the protocol (Table 4), nested selection set $\lambda = 0$ on three folds and $\lambda = 2$ on ResearchCodeBench, where the transfer is wrong-signed, so B4 with nested λ is worse than B2 with the loss entirely from that fold; the fixed $\lambda = 1$ and absolute-level variants are also worse, and every B4 variant has higher ECE than B2 at budgets of 3 and above. In the screening, the best pooled Spearman of any representation is 0.097 (TF-IDF), the frozen encoders reach 0.067 and 0.079, and the best single fold is $+0.21$; no representation clears the gate. Cross-benchmark difficulty prediction from text therefore failed on this pool, and a fitted difficulty model can be wrong-signed on a benchmark outside its text regime, although the mechanism itself is sound and large (Section 5.5). No item model is shipped.

5.3 Acquisition policies (H2)

Under the fixed $m = 2$ model, three policies were run through the full protocol: the platform's random order, uncertainty sampling (label while the current pair estimate is within 0.15 of 0.5, otherwise defer, with a reserve rule that fills the budget at the end of the stream), and eager labelling of the first 31 candidates. Pooled ALC is 0.1956, 0.1976 and 0.1971 respectively; neither alternative is distinguishable from random (Table 4). For a pair-level predictor every candidate is exchangeable, so a policy changes which items are observed but not their expected information content; acquisition has content only when the predictor can distinguish items before labelling them, which requires the item-level information H1 did not obtain. No acquisition policy is shipped.

5.4 Within-pair local outcome transfer (H3)

Even without cross-benchmark transfer, outcomes on labelled items that are semantically similar to the target, within the same pair, might carry information beyond the pair rate. The smoother $\hat{p}_i = (w_0\, p_{\mathrm{B2}} + \sum_j w_{ij} y_j)/(w_0 + \sum_j w_{ij})$ with $w_{ij} = \exp(-(1 - \cos(e_i, e_j))/\tau)$ on frozen bge-base embeddings, with (τ, w_0) chosen from $\{0.05, 0.1, 0.2\} \times \{1, 2, 4\}$ by nested LOBO, is compared with B2 and with a permutation control that shuffles outcomes among the labelled items of the pair.

Table 5: Analysis-only ceilings for item-difficulty information (seed 0, 140 pairs). In-benchmark CV uses the held-out benchmark's own outcomes through five-fold cross-validation; true uses each item's actual relative difficulty; λ is chosen on the held-out fold.

Difficulty source for B4	λ	ALC	B_0	B_7	B_{31}
None (B2 fixed $m = 2$, seed 0)	0	0.1962	0.250	0.187	0.171
In-benchmark CV prediction, 19 text features	0.5	0.1908	0.244	0.183	0.165
True item difficulty (oracle)	1.0	0.1211	0.186	0.105	0.088

Result. The signal exists: with every other item of a pair available as a neighbour, an item's residual outcome correlates 0.14 to 0.24 with the similarity-weighted residual of its neighbours, against a within-pair permutation null near zero. In the protocol H3 beats its own permutation control (-0.0008 $[-0.0011, -0.0005]$, 109 of 140 pairs) but is worse than B2 (Table 4); nested selection chose the most conservative grid corner ($\tau = 0.05$, $w_0 = 4$) on three of four folds. A budget-matched version of the diagnostic recorded in the local result files (31 random labelled neighbours per target; Appendix A) indicates why: the correlation falls to 0.03 to 0.15, and the median cosine similarity to the nearest labelled item is about 0.67 to 0.73 on three benchmarks (1.00 on ResearchCodeBench, where chunk-averaged embeddings of items from the same paper nearly coincide). Reweighting 31 labels unequally costs effective sample size, and the signal at this budget is too weak to pay for it. Not adopted.

5.5 Oracle and requirement analyses

These analyses use information a submission cannot have (seed 0; analysis-only, not submission scores).

Feeding B4 the true relative item difficulty (from all outcomes of the held-out benchmark) with $\lambda = 1$ gives a pooled ALC of 0.1211 against 0.1962 for B2 on the same seed (Table 5); the gain is monotone in λ up to the natural scale 1 and holds on every benchmark (0.119, 0.125, 0.126 and 0.057 against B2's 0.180, 0.208, 0.227 and 0.259). About 7.5 ALC points are available from item difficulty, three times the gain of shrinkage over the baseline; the bottleneck is transferable difficulty estimation, not the Bayesian update. A requirement analysis with noisy true difficulty (Appendix B) places the break-even near Spearman 0.5 (ALC 0.193 at 0.48, 0.160 at 0.67); the deployable representations reached at most 0.21 on any fold and 0.10 pooled, which motivated the gate.

6 Formative Codabench result

The final submission (id 946829, package `B2_beta_mean.zip`) completed the Codabench [12] runtime check successfully with a formative Brier ALC of **0.170341** and mean Brier by budget of 0.250000, 0.201693, 0.165999, 0.145557, 0.143180 and 0.140554 at budgets 0, 1, 3, 7, 15 and 31, which reproduce the ALC through Eq. (1): exactly 0.250 at budget 0, well below 0.250 at budget 1, and monotone improvement with labels, as designed. Our earlier submission of the official empirical-mean baseline (id 943338) scored 0.226339 with a budget-1 Brier of 0.318878, the overconfidence shape seen offline.

The formative score is computed on a small subset of hidden-test pairs (eight pairs in the scoring logs of both submissions) that the platform samples afresh for every submission [8], so the two formative numbers come from different pairs and are not a paired comparison, and the formative 0.170341 is not comparable with the training-pool figures of Section 4. No hidden benchmark examples or response labels were accessed; the aggregate formative feedback served only as a runtime and sanity reference, never for training, hyperparameter selection or model selection.

7 Limitations

- **Four eligible training benchmarks with highly unequal pair counts.** Every conclusion is conditional on this pool; cross-benchmark inference from four benchmarks, one of them a single pair, is thin, and generalisation to unseen benchmark families is uncertain. Multi-SWE-bench contributes 82 of 140 pairs and dominates the pooled selection statistic;

every comparison is repeated without it, but the remaining 58 pairs come from only three benchmarks.

- **Data-limited hyperparameter selection.** LOBO selection of m rests on three training benchmarks per fold; the fold-selected values (1.5 to 3) and the fixed $m = 2$ agree, but the resolution is coarse, and $m = 2$ was chosen from evidence across the whole released benchmark set. The strict LOBO ALC of 0.1972 is the cross-validated estimate; the full-pool 0.1956 is descriptive, not an unbiased held-out estimate.
- **Domain heterogeneity and sparse metadata.** Item text spans three orders of magnitude in length and success rates span 0.15 to 0.48 across benchmarks; text-based difficulty prediction that transfers weakly among the GitHub-issue benchmarks and REAL is wrong-signed for ResearchCodeBench (Appendix A), and only model name overlaps across benchmarks (Section 2).
- **Unstable structured alternatives.** The configuration prior and the item-difficulty models produced pooled gains or losses that depended on which benchmark was held out or removed (Section 5), and were not deployed for that reason, not because the mechanisms are unsound: the oracle shows item difficulty is the largest lever measured here.
- **Deliberate simplicity.** B2 cannot exploit the benchmark's level, the subject's identity or the item's text (Section 5.5 quantifies the cost), and its worst case is the official baseline's behaviour at large budgets; we claim only that on this pool it was the most robust option tested.

8 Reproducibility and released materials

Code release. Source code, experiment scripts, unit tests and the submission source are released at `https://github.com/omuili/probityx-aims-2026`; this report is submitted separately. The released scripts regenerate experiment outputs locally under `reports/`; generated outputs (pair-level CSV files, markdown summaries, logs) are not version-controlled, and neither is the packaged submission archive, which is rebuilt from `submissions/B2_beta_mean/model.py` (Appendix D lists every command). The tests, the B0/B1 baselines and the B2 LOBO selection were re-run from a clean checkout of commit `647176c` ("Initial release") and reproduced the archived outputs to numerical precision (10^{-12}); all 40 tests pass.

Data and environment. Dataset `aims-foundations/measurement-db` at the revision listed below, as recorded in `data/raw/MANIFEST.json`. Main environment: Python 3.11.16, NumPy 2.4.6, pandas 3.0.6, SciPy 1.17.1, scikit-learn 1.9.1, pyarrow 25.0.1. The representation screening and the H3 experiment additionally require `torch` (CPU is sufficient) and `sentence-transformers`, whose versions were not recorded in the release; re-running those two scripts rebuilds an embedding cache under `data/cache/` that is not part of the release. Seeds: split 0; candidate order 0, 1, 2 (Section 2.2 lists the seed-0-only analyses); bootstrap 0.

Submitted model and exact identifiers. Final Codabench submission 946829. The archive was built from the released `model.py` and validated with the organizers' submission checker [9]; a rebuilt archive differs in zip timestamps, so verification should compare the extracted file against the SHA-256 below.

```
measurement-db revision  bc8204d811823da849c6686bf124d4ca9f82e4de
model.py SHA-256         928ea8ca34cfc54780ca0642b6660ca8d2d365ec0a74fe6661afefde22f43054
```

Disclosures. *Training data:* the four eligible `measurement-db` benchmarks at the stated revision and nothing else; no hidden benchmark examples or response labels were accessed, and aggregate formative feedback served only as a runtime check (Section 6). *Pretrained models:* the frozen encoders all-MiniLM-L6-v2 and bge-base-en-v1.5, used only in the closed H1 screening and H3 analyses; the submitted model has none. *Licenses:* `measurement-db` is listed under CC-BY-SA 4.0 and was used under its gated-access terms without being redistributed; the official baseline code is under the MIT license; all-MiniLM-L6-v2 is under Apache-2.0 and bge-base-en-v1.5 under MIT; the remaining dependencies are standard open-source Python packages at the versions listed above. *An LLM-based coding assistant* was used as a development tool. The research, including the experimental design, analysis, interpretation of results, and final submission decisions, was conducted by the author.

References

[1] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[2] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman and Hall/CRC, 3rd edition, 2013.

[3] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[4] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017.

[5] Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danmarks Pædagogiske Institut, Copenhagen, 1960.

[6] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[7] Nhi Truong, Sang T. Truong, and Sanmi Koyejo. The AI measurement data bank. https://aimslab.stanford.edu/measurement-db, 2026. AIMS Lab, Stanford University. Gated Hugging Face dataset `aims-foundations/measurement-db`.

[8] Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. The predictive AI evaluation competition at NeurIPS 2026. Competition website, https://aims.stanford.edu/competition, 2026. Accessed 22 October 2026.

[9] Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, https://github.com/aims-foundations/paiec_baseline, 2026.

[10] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[11] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged resources to advance general Chinese embedding. arXiv:2309.07597, 2023.

[12] Zhen Xu, Sergio Escalera, Adrien Pavão, Magali Richard, Wei-Wei Tu, Quanming Yao, Huan Zhao, and Isabelle Guyon. Codabench: Flexible, easy-to-use and reproducible meta-benchmark platform. *Patterns*, 3(7):100543, 2022.

A Provenance of supplementary diagnostics

Every number in Tables 2–5 is printed by the released scripts, except the "excluding Multi-SWE-bench" intervals for B3, the fixed-λ and absolute-level B4 variants, the acquisition policies and the analysis-only B4 row of Table 4, and the fixed-$m = 2$ bootstrap of Section 4; these were recomputed for this report from the pair-level outputs that the scripts write, with the same bootstrap procedure. The following diagnostics quoted in the text are not printed by the released scripts; each was verified read-only from the released data tables or the scripts' pair-level outputs before inclusion.

- **Constant-within-pair Brier (0.1635).** Mean over the 140 pairs of $\bar{y}(1-\bar{y})$, with $\bar{y}$ each pair's success rate over all its recorded responses as stored in the pair-level outputs.
- **B3-global prior means and benchmark-level miss.** $\sigma(G)$ refitted per fold from the training benchmarks' subject tables with the released `ConfigPrior` class (0.366, 0.307, 0.301, 0.257), compared with the mean success rate of the held-out benchmark's eligible pairs (0.220, 0.340, 0.353, 0.479).

- **Ability transfer.** For each benchmark, the mean subject logit of each model name, centred within benchmark; Spearman correlation on the model names shared between two benchmarks (11, 9 and 13 shared names).
- **Harness share of variance.** On the 82 Multi-SWE-bench pairs, between-group sum of squares over total sum of squares of the pair success rate grouped by harness (0.771 on the logit scale, 0.760 on the probability scale) and by model name (0.214, probability scale).
- **Pairwise difficulty transfer.** The ridge difficulty model (penalty 10) fitted on the items of one benchmark and tested on another; Spearman correlation with true relative difficulty (rows: training benchmark; columns: test benchmark, in the order Multi-SWE-bench, REAL, ResearchCodeBench, SWE-rebench): Multi-SWE-bench $(\cdot, +0.20, -0.13, +0.14)$; REAL $(-0.05, \cdot, -0.18, +0.01)$; ResearchCodeBench $(-0.05, -0.34, \cdot, -0.01)$; SWE-rebench $(+0.24, +0.38, -0.34, \cdot)$. Transfer from either GitHub-issue benchmark to the other or to REAL is weakly positive ($+0.14$ to $+0.38$); every pairing in which ResearchCodeBench is the training or test benchmark is negative (-0.01 to -0.34); training on REAL transfers to neither software benchmark.
- **Feature-sign stability.** Spearman correlation of each of the 19 text statistics with relative difficulty inside each benchmark: no feature has the same sign on all four benchmarks with $|\rho| \geq 0.10$ on at least three. Text length is the strongest single feature on REAL ($\rho = -0.40$; longer web tasks are harder) and is near zero or reversed elsewhere.
- **Budget-matched H3 diagnostic.** The correlations of 0.03 to 0.15 and the nearest-neighbour cosine similarities quoted in Section 5.4 are taken from the local result file of the H3 experiment. They are not regenerated by the released scripts and could not be recomputed for this report because the embedding cache is not part of the release; they are reported as supporting context only, and no conclusion depends on them.

B Requirement curve

Item difficulty equal to the true relative difficulty plus Gaussian noise tuned to a target Spearman correlation, fed to B4 (seed 0, 140 pairs). Row "none" is B2 fixed $m = 2$ on the same seed. At each target the better of $\lambda \in \{0.5, 1\}$ is shown.

Spearman achieved	λ	ALC	B_0	B_1	B_7
none (B2)	0	0.196	0.250	0.214	0.187
0.30	0.5	0.248	0.303	0.277	0.232
0.48	0.5	0.193	0.247	0.220	0.178
0.67	0.5	0.160	0.217	0.177	0.149
0.84	1.0	0.137	0.199	0.162	0.121

Adding noise inflates the variance of the difficulty signal (at Spearman 0.30 the noisy signal has about eleven times the variance of the truth), whereas a fitted predictor with the same correlation is shrunk toward zero, so the curve overstates the harm at low correlation and the break-even lies somewhat below 0.48.

C Note on a corrected sign convention

During development, one run of the B4 analysis passed the predicted *success* logit where the model expected a *difficulty* (higher is harder), inverting the item term; it made true difficulty appear harmful, which is what exposed the error. All B4, oracle and requirement results in this report come from the corrected rerun; two unit tests pin the sign convention, and the B4 likelihood uses a numerically stable log-sigmoid with predicted difficulty bounded to ± 6 logits.

D Reproduction commands

All commands run from the repository root; comments name the outputs each script writes under `reports/`.

```
uv venv .venv --python 3.11
uv pip install --python .venv/bin/python \
  numpy==2.4.6 pandas==3.0.6 pyarrow==25.0.1 \
  scipy==1.17.1 scikit-learn==1.9.1 \
  huggingface_hub pytest matplotlib tabulate
.venv/bin/hf auth login                                  # measurement-db is gated
# Fetch the exact dataset revision used in this report. Do not run
# paiec.data.download, which resolves the current main branch.
.venv/bin/python - <<'EOF'
from huggingface_hub import snapshot_download
snapshot_download("aims-foundations/measurement-db", repo_type="dataset",
    revision="bc8204d811823da849c6686bf124d4ca9f82e4de", local_dir="data/raw",
    allow_patterns=[f"{b}/{t}.parquet" for b in ("multi_swebench",
        "real_webagents", "researchcodebench", "swe_rebench")
        for t in ("benchmarks", "subjects", "items", "response")])
EOF
.venv/bin/python -m paiec.data.report                    # data report
.venv/bin/python -m pytest -q tests                      # 40 passed
.venv/bin/python scripts/run_baselines.py                # B0, B1
.venv/bin/python scripts/phase2_b2.py                    # B2 LOBO selection of m
.venv/bin/python scripts/phase2_b3.py                    # B2 fixed m=2, B3, B3-global
.venv/bin/python scripts/phase3_step2_difficulty.py      # H1: text -> difficulty
.venv/bin/python scripts/phase3_step4_b4.py              # H1: B4 in the protocol
.venv/bin/python scripts/phase3_step4b_ceiling.py        # oracle (analysis only)
.venv/bin/python scripts/phase3_step4c_requirement.py    # requirement curve
.venv/bin/python scripts/phase3_step5_screen.py          # H1 screening (torch)
.venv/bin/python scripts/phase4_h2_pairlevel.py          # H2 policies
.venv/bin/python scripts/phase4_h3_local.py              # H3 (torch, cache)
.venv/bin/python scripts/final_summary.py                # summary table, figure
(cd submissions/B2_beta_mean && zip -j ../../dist/B2_beta_mean.zip model.py)
git clone https://github.com/aims-foundations/paiec_baseline \
    third_party/paiec_baseline
.venv/bin/python third_party/paiec_baseline/check_submission_zip.py \
    dist/B2_beta_mean.zip
```

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the method, the evaluation protocol and the scope of the claims; the supporting results are in Sections 4 and 5, and the abstract separates the cross-validated estimate from the descriptive full-pool figure.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 is a dedicated limitations section; Section 4 also reports where the method loses to the baseline.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theorems or formal theoretical results; its equations define the metric and the estimator.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 2.2 and 3 specify the protocol and method, and Section 8 and Appendix D give the dataset revision, seeds, software versions and commands. Appendix A identifies the supporting diagnostics that the released scripts do not regenerate.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code and exact commands are provided through the repository in Section 8 and Appendix D. The data are the competition-provided `measurement-db`, which is public but access-gated on Hugging Face, at the stated revision; the embedding cache and generated outputs are not released and Appendix A lists the one diagnostic that consequently cannot be recomputed.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 2.2 gives the splits, budgets, seeds and nested selection; Sections 3 and 5 give the hyperparameter grids and how values were chosen. The submitted estimator uses no optimizer.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Section 2.2 defines the paired bootstrap over pairs (5,000 resamples, 95% percentile intervals); intervals are reported in Section 4 and Table 4.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: All experiments ran on CPU on one laptop under WSL (Intel Core i9-14900HX, 32 threads, 15 GB RAM available to WSL), with no GPU, cloud or cluster resources. Storage: 56 MB of data, 42 MB of embedding cache and 9 MB of outputs. Measured time per experiment: baselines 11 s, B2 selection 36 s, B3 3.9 min, text-difficulty fitting 5.9 min, B4 protocol 24.6 min, oracle 9.0 min, requirement curve 4.7 min, representation screening including encoding 59.6 min, H2 12 s, H3, using the saved embeddings 11.3 min. Summary and figure scripts take about 2 s. Total about 2.0 hours; peak memory was 3.2 GB (representation screening) and at most 1.2 GB otherwise. The whole project, including the superseded B4 run noted in Appendix C, ran on this laptop.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work involves no human subjects, used only the competition-provided data and accessed no hidden labels, and its data and tool use are disclosed in Section 8.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [No]

    Justification: The paper is a technical report on a competition prediction method and does not discuss societal impacts; the submitted model is a simple statistical estimator with no direct path to a harmful application identified by the author.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: No high-risk data or models are released: the submitted model is a small standard-library Python file with two constants, and no dataset is redistributed.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Creators of the data, code and models used are cited, and Section 8 states the dataset revision, software versions and the license of each asset, and that the dataset was used under its gated-access terms without being redistributed.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new asset is research code, documented by the repository README and by Section 8 and Appendix D; the repository declares the MIT license. No new dataset or pretrained model is introduced.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects; the responses analysed are runs of AI systems.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not a component of the core method. An LLM-based coding assistant used as a development tool is disclosed in Section 8, and frozen text encoders appear only in closed analyses that were not submitted.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.